Daftar — v1 Build Spec & Style Guide

Oct 8, 2026 · @Justin Andreasen

Decision log (changes vs. spec-brief.md)

These decisions were made after the original brief and override it. Copy them into spec-brief.md so the knowledge file stays the source of truth.

Topic	Decision
Price	$49 one-time unlock. A $2.99/mo alternative will be evaluated later; it conflicts with the "own forever" tagline.
Photo storage	2 GB photo cap per account, included in the unlock.
Burn mode	Moved to v2.
Chapters	Flat in v1 (no nesting). Nesting moves to v2.
Prompt library	English only. Content is written outside the build; the app ships the schema plus a seed file.
Sealed letters	Exported in full with `sealed: true` and the unlock date. The seal is a UI promise, not encryption.
Arabic / RTL	Minimal layer kept: per-paragraph auto direction, an Arabic-capable font stack, per-entry language tag + filter. Cut: Arabic search normalization, transliteration, non-English prompts.
Device priority	Phone and desktop are equal first-class targets.
Never lose text	Local draft buffer (IndexedDB) in v1, in addition to server autosave.
Design	"Lamplight" direction, built in CSS. No photographic or AI-generated backgrounds in the app.

1. Product summary and target users

Daftar is "the journal you actually own": a calm, lamplit writing app where every word can leave with you as a readable JSON file at any moment. It is a PWA that works equally well on phone and desktop. It has no AI, no streaks, and no lock-in.

What makes it different

- **Ownership you can see.** The Vault shows when you last backed up and exports everything in one tap. Reading and exporting stay free forever.
- **Organization that matches a life.** Tracks (what kind of writing), tags (what it's about), and chapters (when in life).
- **Writing in any language.** Arabic and other RTL scripts flow correctly, even when mixed with English in one entry.
- **Letters to the future.** Sealed letters stay closed until their date.

Target users

Who	What they need from Daftar
Lapsed journalers burned by app shutdowns or lock-in	Export they trust, a one-time price, no subscription fatigue
Spiritual journalers	A Spiritual track with scripture, impressions, and "answered later?" follow-ups
Multilingual writers and language learners	Journaling in a second language, RTL that works, filtering by language
Privacy-conscious minimalists	An honest privacy policy, no AI, no data mining, a plain file they can read

Not for (v1): teams, shared journals, therapists' clients needing E2E encryption, or heavy photo-diary users (2 GB cap).

2. Tech stack

Use a static React SPA (installable PWA) on Cloudflare Pages, Supabase for everything server-side, Tiptap for the editor, and a merchant-of-record payment provider. Fixed cost at launch is about $25/month. Prices below are approximate from memory and must be verified before launch.

Layer	Choice	Why	Approx. cost
Frontend	Vite + React + TypeScript	No SEO needs inside the app, so no server rendering. Claude Code is strongest in React.	$0
PWA	vite-plugin-pwa (Workbox)	Install to home screen, cache the app shell, offline reading of the loaded session	$0
Styling	Plain CSS with custom properties (design tokens) + CSS modules	Lamplight needs hand-tuned textures and tokens; utility frameworks push toward the generic look	$0
Editor	Tiptap (ProseMirror)	Mature, headless (we own every pixel), JSON document model that maps cleanly to export, block-level `dir` attributes for RTL	$0 (open-source core)
Local buffer	IndexedDB via `idb-keyval`	Drafts survive tab crashes, dead batteries, and café Wi-Fi	$0
Backend	Supabase (Auth, Postgres, Storage, Edge Functions)	One vendor, RLS built in, encryption at rest by default	Free tier for development; Pro about $25/mo at launch
Hosting	Cloudflare Pages	Free, fast global CDN, unlimited static bandwidth	$0

Layer	Choice	Why	Approx. cost
Payments	Lemon Squeezy or Paddle (merchant of record)	They handle sales tax and VAT worldwide, which matters for a solo founder selling globally	About 5% + $0.50 per sale, so a $49 sale nets about $46
Image compression	`browser-image-compression` (client side)	No server image processing costs	$0
Search	Postgres full-text search + `pg_trgm`	No external search service	$0
Error tracking	Sentry free tier	Know when autosave fails before users tell you	$0
Email	Supabase Auth email at launch, Resend later if volume grows	Magic links and receipts only	$0 at launch

Why not Stripe directly? Stripe's fees are lower, but you become responsible for tax collection in every jurisdiction. For a $49 one-time product sold worldwide, the merchant-of-record fee is cheaper than an accountant.

Why not Next.js? The marketing page can be a separate static page. The app itself is entirely behind login, so server rendering adds complexity and hosting cost for no benefit.

3. Data model, Supabase schema, and RLS

Ten tables (including `entry_versions`, described in section 5), all owned per user through `user_id = auth.uid()`, plus one read-only global table for prompts. Entry IDs are UUIDs generated on the client, which makes autosave, offline drafts, and merge-by-ID safe.

Design rules

- Every user table has `id uuid`, `user_id`, `created_at`, `updated_at`, and `deleted_at` (soft delete). Hard deletes run only from the account-deletion function.
- `updated_at` is set by a trigger, never trusted from the client. Merge compares it.

- Licensing lives in its own table that only the service role can write. Users can never grant themselves an unlock through the API.
- Chapters are **not** joined to entries. An entry is in a chapter when its `entry_date` falls inside the chapter's range, which is computed at query time.

Tables

Table	Key columns	Notes
`profiles`	`id` (= auth user id), `display_name`, `timezone`, `default_language`, `last_export_at`, `storage_bytes_used`	One row per user, created by a trigger on signup
`licenses`	`user_id` (PK), `trial_started_at`, `trial_ends_at`, `status` (`trial` / `unlocked`), `unlocked_at`, `provider`, `provider_order_id`	Service-role writes only; users can read their own row
`tracks`	`name`, `slug`, `color_token`, `default_template_id`, `sort_order`, `is_builtin`	Built-in tracks are copied into each new account so users can rename them
`templates`	`name`, `track_id`, `body_rich` (Tiptap JSON), `is_builtin`	`user_id` null = global built-in, read-only
`entries`	`track_id`, `title`, `entry_date` (date), `body_rich` (jsonb), `body_text` (text), `language` (BCP-47, e.g. `en`, `ar`), `word_count`, `fields` (jsonb), `is_letter`, `letter_to`, `sealed`, `unlock_at`, `revision`	`fields` holds template data such as `answered_later` and `answered_on`
`tags`	`name`	Unique on `(user_id, lower(name))`
`entry_tags`	`entry_id`, `tag_id`, `user_id`	`user_id` duplicated so RLS stays a single equality check
`chapters`	`name`, `description`, `start_date`, `end_date` (nullable = ongoing), `color_token`	Flat in v1; add `parent_id` in v2

Table	Key columns	Notes
`media`	`entry_id`, `storage_path`, `mime`, `bytes`, `width`, `height`, `sha256`	`sha256` deduplicates on restore
`prompts` (global)	`text`, `depth` (light / medium / deep), `mood`, `time_of_day`, `track_hint`, `language`	Seeded from a JSON file; public read, no write

Core SQL (entries, indexes, licensing check)

```
create extension if not exists pg_trgm;

create table entries (
  id uuid primary key,                    -- generated client-side
  user_id uuid not null references auth.users on delete cascade,
  track_id uuid references tracks(id),
  title text,
  entry_date date not null,
  body_rich jsonb not null default '{}',
  body_text text not null default '',
  language text not null default 'en',
  word_count int not null default 0,
  fields jsonb not null default '{}',
  is_letter boolean not null default false,
  letter_to text,
  sealed boolean not null default false,
  unlock_at timestamptz,
  revision int not null default 1,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  deleted_at timestamptz,
  search tsvector generated always as (
    to_tsvector('simple', coalesce(title,'') || ' ' || body_text)
  ) stored,
  constraint sealed_needs_date check (not sealed or unlock_at is not null)
);

create index entries_user_date on entries (user_id, entry_date desc) where
deleted_at is null;
create index entries_search on entries using gin (search);
create index entries_trgm on entries using gin (body_text gin_trgm_ops);
create index entries_lang on entries (user_id, language);
```

```
-- Can this user write? Trial still running, or unlocked.
create function can_write() returns boolean
language sql stable security definer set search_path = public as $$
  select exists (
    select 1 from licenses
    where user_id = auth.uid()
      and (status = 'unlocked' or trial_ends_at > now())
  );
$$;
```

RLS policies (same pattern on every user table)

```
alter table entries enable row level security;

create policy entries_read on entries for select
  using (user_id = auth.uid());                    -- reading is free
forever

create policy entries_insert on entries for insert
  with check (user_id = auth.uid() and can_write());

create policy entries_update on entries for update
  using (user_id = auth.uid() and can_edit())   -- trial + 48h grace, see
section 9
  with check (user_id = auth.uid());

-- No delete policy: deletes are soft (update deleted_at).

alter table licenses enable row level security;
create policy licenses_read on licenses for select using (user_id =
auth.uid());
-- No insert/update policies: only Edge Functions with the service role write
here.

alter table prompts enable row level security;
create policy prompts_read on prompts for select using (true);
```

Storage

- One private bucket, `media`. Paths are `{user_id}/{entry_id}/{media_id}.webp`.
- Policy: read when `(storage.foldername(name))[1] = auth.uid()::text`. Upload requires the same plus `can_write()`.

- Photos are served through signed URLs that expire after 1 hour.

Search notes

- The `simple` text-search config does no stemming, so it treats English and Arabic tokens the same. That is acceptable for v1.
- Trigram search covers partial words and typos. Arabic normalization (harakat, alef forms) is deferred, per the decision log.

4. Export JSON schema

The export is one UTF-8 JSON file, `daftar-journal-YYYY-MM-DD.json`, that a stranger, a script, or an AI can understand with no documentation. The .zip variant holds the same file plus a `/media` folder.

Format rules

- Top-level `_readme` explains the file in plain English.
- `format` and `schema_version` (integer, starting at 1) allow future migrations. Restore refuses versions newer than it understands and upgrades older ones.
- Every entry carries `body_text` (plain text, always readable) **and** `body_rich` (Tiptap/ProseMirror JSON, for perfect round-trips).
- References use IDs (`track_id`, `tag_ids`), and the referenced objects are included in the same file, so the file is complete on its own.
- Dates are ISO 8601. `entry_date` is a calendar date with no time zone; timestamps are UTC.
- Soft-deleted entries are excluded. Sealed letters are included in full with `sealed: true`.
- Media appear as metadata with a relative `file` path. In the text-only export the path points to a file that isn't present, and the `_readme` says so.

Example file (abbreviated)

```
{
  "_readme": "This is your complete Daftar journal. Each item in 'entries' is
one journal entry. 'body_text' is the plain text you wrote; 'body_rich' is
the same text with formatting, in ProseMirror JSON. Tracks, tags, and
chapters are listed separately and referenced by id. Dates use ISO 8601
(YYYY-MM-DD). Photos are listed under each entry; in a .zip export they are
in the /media folder. You can read this file in any text editor. It belongs
```

```
to you.",
  "format": "daftar.journal",
  "schema_version": 1,
  "exported_at": "2026-10-08T11:20:00Z",
  "app_version": "1.0.0",
  "export_type": "full_zip",
  "account": {
    "display_name": "Yasin",
    "timezone": "America/Denver",
    "default_language": "en"
  },
  "tracks": [
    { "id": "6f1c…", "name": "Spiritual", "slug": "spiritual", "color_token":
"olive", "sort_order": 2 }
  ],
  "tags": [
    { "id": "a91d…", "name": "family" }
  ],
  "chapters": [
    { "id": "c22e…", "name": "Junior year", "start_date": "2026-09-01",
"end_date": null, "description": "" }
  ],
  "entries": [
    {
      "id": "0b7e5f2a-…",
      "entry_date": "2026-10-07",
      "title": "Morning by the window",
      "track_id": "6f1c…",
      "tag_ids": ["a91d…"],
      "language": "en",
      "body_text": "Read Alma 32 this morning. The seed comparison landed
differently today.\n\nالحمد لله على كل حال",
      "body_rich": { "type": "doc", "content": [ { "type": "paragraph",
"content": [ { "type": "text", "text": "Read Alma 32 this morning…" } ] } ]
},
      "word_count": 19,
      "fields": { "scripture": "Alma 32", "answered_later": false },
      "letter": null,
      "media": [
        { "id": "m3…", "file": "media/0b7e5f2a/m3.webp", "mime":
"image/webp", "width": 1600, "height": 1200, "bytes": 184233, "sha256": "9f…"
}
      ],
```

```
      "created_at": "2026-10-07T14:02:11Z",
      "updated_at": "2026-10-07T14:19:40Z"
    },
    {
      "id": "4c09…",
      "entry_date": "2026-10-08",
      "title": "To me, at graduation",
      "track_id": "…letters…",
      "tag_ids": [],
      "language": "en",
      "body_text": "…",
      "body_rich": { "type": "doc", "content": [] },
      "word_count": 412,
      "fields": {},
      "letter": { "to": "Future me", "sealed": true, "unlock_at": "2029-04-
20T15:00:00Z" },
      "media": [],
      "created_at": "2026-10-08T09:00:00Z",
      "updated_at": "2026-10-08T09:30:00Z"
    }
  ]
}
```

The app also generates a short `README.txt` in the .zip with the same text as `_readme`, for anyone who opens the folder before the file.

5. Restore and merge

Restore is a two-step, idempotent merge: a dry-run preview the user confirms, then an apply that can be safely re-run. It never creates duplicates and never destroys text. Any version it replaces is kept in `entry_versions`.

Flow

1. **Read and validate.** Parse the JSON (or unzip). Check `format = daftar.journal` and `schema_version`. Newer versions are refused with a clear message; older versions are upgraded in memory.
2. **Dry run.** Compare everything against the account and show a report: new entries, updated entries, unchanged, skipped (local is newer), restored from trash, photos to upload, and photos skipped by the cap.
3. **Confirm.** One button: "Merge into my journal." Nothing is written before this.

4. **Apply.** Send batches of 100 entries to a Postgres function, `restore_batch(jsonb)`, that runs with the caller's permissions, so RLS still applies. Then upload missing photos.
5. **Report.** Show a summary, and offer a download of the report as a text file.

Conflict rules

Situation	Rule
Entry ID not in account	Insert it, keeping the original ID.
Entry ID exists, content identical (hash of title, body_text, fields, tags, letter, date)	Skip.
Entry ID exists, imported `updated_at` is newer	Copy the current row into `entry_versions`, then overwrite it.
Entry ID exists, local `updated_at` is newer or equal	Keep local; list as "skipped, yours is newer."
Entry is soft-deleted locally, imported copy is newer than `deleted_at`	Undelete and update it.
Entry is soft-deleted locally, deletion is newer	Stay deleted; list it in the report.
ID belongs to another account (file restored into a different account)	Insert fails on the primary key. Generate a new ID, record the remap, and continue.
Track, tag, or chapter ID matches	Keep local name and settings; newer `updated_at` wins for renames.
No ID match, but the name matches (case-insensitive)	Reuse the local item and remap the imported ID.
No match at all	Create it, keeping the imported ID.
Tags on a merged entry	Union of local and imported tags.
Photo with matching `id` or `sha256` already stored	Skip the upload.
Photo would exceed the 2 GB cap	Import the text anyway; list skipped photos so the user keeps the .zip.
Sealed letter	Letter fields follow the same newer-wins rule. The seal is never broken by restore.

`entry_versions` (small extra table)

Columns are `id`, `entry_id`, `user_id`, `snapshot` (jsonb of the full row), `reason` (`restore_overwrite`), and `created_at`. Versions older than 90 days are pruned. They also appear in the export under `entry_versions` so nothing is ever only on the server.

Permission: restore needs an active trial or unlock, since it writes entries. See Open questions.

6. Photo pipeline

All image processing happens in the browser. Photos are compressed to WebP at 1600 px on the long edge (about 150–300 KB each) before upload, which makes the 2 GB cap roughly 8,000 photos.

Steps

1. User picks or pastes an image (JPEG, PNG, HEIC where the browser can decode it, WebP).
2. Read EXIF orientation, rotate, then **strip all EXIF metadata**, including GPS. Privacy by default.
3. Resize to a maximum of 1600 px on the long edge and encode WebP at quality 0.8. Also make a 400 px thumbnail.
4. Compute SHA-256 of the compressed file for deduplication.
5. Check the cap: `storage_bytes_used + new bytes <= 2 GB`. If over, block the upload with a friendly message and keep the text.
6. Insert a placeholder node in the editor immediately, upload in the background, and swap in the real image when done. If the upload fails, keep the image in IndexedDB and retry.
7. Write the `media` row. A trigger updates `profiles.storage_bytes_used`.

Limits

Limit	Value
Original file accepted	Up to 25 MB
Stored size	1600 px long edge, WebP
Photos per entry	12
Account cap	2 GB (thumbnails count)
Signed URL lifetime	1 hour

Deletion: removing a photo from an entry soft-deletes its `media` row. A nightly job deletes storage objects that have been soft-deleted for 30 days, then recalculates usage. HEIC decoding is uneven across browsers; when it fails, show "This photo format isn't supported here. Try exporting it as JPEG."

7. Screens and user flows

V1 has nine screens. Every flow is designed so that writing is never more than one tap away and nothing ever interrupts a sentence.

Navigation: on phone, a bottom bar with Desk, Browse, Write (centre, larger), Letters, and Vault. On desktop, a slim left rail with the same items. Settings sits behind the account initial.

Screen	Purpose	Key elements
Onboarding	Get to the first sentence in under 60 seconds	Magic-link sign-in, then three steps: your name; pick tracks (Daily, Spiritual, Letters, Dreams pre-checked); default language. Ends in the editor with a gentle first prompt. Trial starts here.
Desk (home)	The day at a glance	Today's entry ("Write today" or "Keep going"), dice-roll prompt, last 5 entries, quiet backup line ("Last backup: 12 days ago")
Write	The editor	Parchment page; header with date, track, language, and tags; minimal toolbar (bold, italic, heading, lists, photo); save status line
Browse	Find anything	Calendar with dots on days with entries; list view; filters for track, tag, language, chapter, date range; search box
Read	"Read like a book"	One day per spread; swipe on phone, arrow keys on desktop; empty runs collapse to "3 quiet days"; jump-to-date

Screen	Purpose	Key elements
Chapters	Name parts of life	List of named date ranges; create with name, start, optional end (ongoing); tapping opens Read filtered to that range
Letters	Write to someone, or to later	Compose with a "To" field; seal toggle plus unlock date. Sealed letters show as a wax-sealed card: "Opens 20 April 2029."
Vault	Ownership, made visible	Export JSON, export .zip, restore, last backup date, storage used out of 2 GB
Settings	Account and preferences	Name, default language, theme (Lamplight / Daylight), opt-in reminders, unlock status, delete account (offers export first)

Never lose text: autosave spec

1. Every change writes the full draft to IndexedDB after 500 ms of no typing, keyed by entry ID.
2. The server save runs after 2 seconds idle, and at least every 5 seconds during continuous typing.
3. Saves flush immediately on blur, `visibilitychange` (hidden), route change, and `pagehide`.
4. Each save sends the `revision` it was based on. If the server's revision is newer (another tab or device), the server row is copied to `entry_versions` (reason `concurrent_edit`) and the new save wins. The user sees a small note: "This entry was also edited elsewhere. The other version is kept."
5. Offline or failing saves queue in IndexedDB and retry with backoff. The buffer clears only after the server confirms.
6. On app open, any unsynced local drafts are pushed first.
7. Status line, always visible and never animated aggressively: "Saving…", "Saved", "Saved on this device. Will sync when you're back online."
8. Tabs coordinate through `BroadcastChannel` so two open tabs don't fight over one entry.

Template flow: creating an entry in a track with a default template pre-fills the body. Spiritual template fields: Scripture, What I noticed, Impressions, Answered later? (checkbox plus date). Ticking "Answered later?" on an old entry is how follow-ups happen. There's no reminder, no nag.

Backup nudge: if the last export is older than 30 days, Desk shows one quiet line with an Export button. It is never a modal, never a badge, and it can be dismissed for 30 days.

Language flow: the language picker sits in the entry header and defaults to the account default. Text direction is automatic per paragraph, so the picker is for organizing, not layout. Browse can filter by language.

8. Style guide: Lamplight

Lamplight is the feeling of writing on parchment at a dark walnut desk, with an oil lamp just out of frame. It is built entirely in CSS and SVG, with no photographs, so it stays sharp, light, and original. There are two themes: **Lamplight** (dark, default) and **Daylight** (the same desk by a window).

Design principles

- **The page is the hero.** The writing surface is always a parchment page sitting on the desk. Chrome stays dim and out of the way.
- **Light has a source.** One warm glow, top-left, as if from a lamp. Highlights and the focus ring use flame amber, so the eye follows the light.
- **Objects, not widgets.** Sealed letters have a wax seal. The dice prompt is a small brass die. Calendar days with entries carry a dot of flame.
- **Quiet by default.** No animation except purposeful transitions under 200 ms. No confetti, no sparkles, no badges.

Anti-template checklist (reject in review if present)

- [ ] No glassmorphism, no purple or blue gradients, no gradient buttons
- [ ] No emoji or sparkle icons in the UI
- [ ] No pure white (#FFF) or pure black (#000) anywhere
- [ ] No default Inter/Tailwind look; every surface uses the tokens below
- [ ] Icons: Phosphor, Light weight, at 20 px; never mixed with another set
- [ ] Rounded corners only on UI controls (pills, buttons). Paper has crisp corners.

Colour tokens

Token	Lamplight (dark)	Daylight (light)	Use
`--desk`	#17110C	#E9DFCB	App background base
`--desk-raised`	#221912	#F1E8D6	Rail, bars, cards on the desk
`--desk-line`	#3A2B1F	#D6C7AB	Dividers on the desk
`--page`	#E4D3B0	#FBF6EA	Writing surface
`--page-line`	#CBB68F	#E7DCC4	Rules and borders on the page
`--ink`	#2A1F16	#2B241C	Text on the page
`--ink-soft`	#6B5640	#6E604E	Secondary text on the page
`--ash`	#DCCBB0	#3B3127	Primary text on the desk
`--ash-soft`	#A38F74	#786A57	Secondary text on the desk
`--flame`	#E2A24C	#A8641C	Accent, focus ring, primary button, calendar dots
`--flame-glow`	rgba(240,168,80,0.16)	rgba(255,214,150,0.35)	Lamp glow, hover warmth
`--olive`	#8A9663	#5E6A40	Secondary accent, Spiritual track, success
`--seal`	#8C2F22	#8C2F22	Wax seal, destructive actions

All text pairs must meet WCAG AA (4.5:1). `--flame` text on `--desk` and `--ink` on `--page` both pass; verify every new pair.

The desk background (CSS recipe)

```
body {
  background-color: var(--desk);
  background-image:
    radial-gradient(ellipse 70% 55% at 18% 12%, var(--flame-glow),
transparent 70%), /* lamp */
    radial-gradient(ellipse 120% 90% at 50% 50%, transparent 55%,
rgba(0,0,0,.45)),   /* vignette */
    repeating-linear-gradient(178deg, rgba(255,220,180,.018) 0 2px,
transparent 2px 9px), /* wood grain */
    url("/grain.svg");
/* fine noise */
  background-attachment: fixed;
}
```

`grain.svg` is a 200×200 SVG using `feTurbulence` (baseFrequency 0.9, 2 octaves) at 5% opacity. It weighs under 1 KB. In Daylight, the vignette drops to 0.12 and the glow becomes soft window light.

The page

- Background `--page` with the same grain at 4% and a very faint warm wash toward the top-left.
- Shadow: `0 1px 0 rgba(0,0,0,.25), 0 24px 48px -16px rgba(0,0,0,.6)`.
- Crisp corners (radius 2 px). Max text width 66 characters. Padding 48 px on desktop, 20 px on phone (the page goes nearly edge to edge).

Typography

Role	Latin	Arabic	Size and line height
Writing and reading	Literata (variable)	Noto Naskh Arabic	19 px / 1.7 desktop, 18 px / 1.7 phone
Headings in entries	Literata, 600	Noto Naskh Arabic, 700	28 / 22 px
Display (entry date, screen titles)	Literata, optical size display, italic	Noto Naskh Arabic	36 / 28 px
UI (buttons, labels, nav)	Source Sans 3	IBM Plex Sans Arabic	15 px / 1.4; small 13 px

Role	Latin	Arabic	Size and line height
Numerals and dates in UI	Source Sans 3, tabular figures	Same	Same as UI

The type scale is 13, 15, 17, 19, 22, 28, 36. All fonts are open-source and self-hosted as WOFF2 subsets. Fonts are listed Latin first, Arabic second, so the browser picks Arabic glyphs automatically. Load Arabic faces with `unicode-range` and `size-adjust: 112%` so Arabic doesn't look smaller than the Latin text beside it.

RTL and mixed text

- Every editor block gets `dir="auto"`, so each paragraph takes its direction from its first strong character. Pair this with `unicode-bidi: plaintext`.
- Inputs (title, tags, search, letter "To") also use `dir="auto"`.
- Use CSS logical properties everywhere (`margin-inline-start`, `padding-inline`, `text-align: start`). No `left` or `right` in layout code.
- Arabic paragraphs get `line-height: 1.9` through `:dir(rtl)`, since Naskh needs more vertical room.
- The app interface stays LTR in v1. Only content flips. An Arabic UI is v2.

Spacing and shape

- Spacing scale (4 px base): 4, 8, 12, 16, 24, 32, 48, 64.
- Radius: 2 px paper, 8 px buttons and inputs, 999 px pills.
- Touch targets at least 44 px. Focus ring: 2 px `--flame` with a 2 px offset, always visible on keyboard focus.

Signature components

Component	Description
Page	The parchment writing and reading surface described above
Track pill	Small pill with a 6 px dot in the track colour; tap to change
Language pill	Shows the language code (EN, AR); tap to change

Component	Description
Brass die	The dice-roll prompt button: a small isometric die in `--flame` line art. Rolls with a 150 ms tilt; reduced motion turns this off.
Wax seal	CSS circle in `--seal` with an embossed دفتر monogram (SVG) and a soft highlight; shows the unlock date beneath
Calendar	Month grid on the desk; days with entries carry a `--flame` dot, chapters show as a thin `--olive` underline across their range
Save status	One line of `--ash-soft` text under the page header; never a toast
Primary button	`--flame` fill, `--ink` text, 8 px radius, no gradient
Quiet button	Text only, `--ash` with an underline on hover

Motion: page turns in Read mode slide 24 px with a fade, 180 ms ease-out. Nothing loops. `prefers-reduced-motion` removes all transforms.

Voice and microcopy

The voice is a warm, literate friend who respects your time. It's plain, a little wry, and never cute, preachy, or guilt-inducing.

Moment	Write this	Never this
Empty Desk	The page is blank. That's allowed.	Start your journaling journey!
Save status	Saved · Saved on this device	All changes saved successfully ✓
Missed days	(nothing)	You broke your 12-day streak
Dice prompt	Roll for a prompt	Need inspiration? ✨
Export done	Your journal is in your hands. 1,204 entries, 3 MB.	Export completed

Moment	Write this	Never this
Sealed letter	Sealed until 20 April 2029.	This content is locked
Trial ending	Your trial ends in 3 days. Everything you've written stays yours either way.	Don't lose access! Upgrade now!
Trial over	Your trial has ended. You can still read and export everything. Unlock Daftar for $49, once, to keep writing.	Your account is restricted
Delete entry	Delete this entry? It moves to the trash for 30 days.	Are you sure?
Save failed	Couldn't reach the server. Your words are safe on this device.	Error 503

9. Trial and licensing logic

The server is the only judge of whether a user can write: the `can_write()` check in RLS. The client only displays state. A trial lasts 21 days from signup, the unlock is $49 once, and reading plus export never depend on either.

States

State	Condition	Read and export	Create entries	Edit existing	Photos
Trial	`status = trial` and `trial_ends_at > now()`	Yes	Yes	Yes	Yes
Grace	Trial ended less than 48 hours ago	Yes	No	Yes (lets in-progress text land)	No
Read-only	Trial ended more than 48 hours ago, or refunded	Yes	No	No	No
Unlocked	`status = unlocked`	Yes	Yes	Yes	Yes

The grace window is implemented as a second function, `can_edit()`, used only by the entries update policy.

Lifecycle

1. **Signup:** a trigger on `auth.users` inserts `profiles` and `licenses` (`status = trial`, `trial_ends_at = now() + 21 days`). Server time only, never the device clock.
2. **Trial messaging:** nothing until 5 days remain. Then one quiet line on Desk with the days left. No emails unless the user opted in.
3. **Checkout:** the Unlock button opens the provider's hosted checkout, passing `user_id` as custom data. No card details ever touch our code.
4. **Webhook:** an Edge Function verifies the provider's signature, then sets `status = unlocked`, `unlocked_at`, and `provider_order_id`. It is idempotent: the same order ID twice does nothing.
5. **Confirmation:** the client listens for the license row change (Supabase Realtime), or polls for up to 60 seconds, then shows "Daftar is yours. Write as long as you like."
6. **Refund:** the refund webhook sets `status = refunded`, which behaves like read-only. No data is touched.

Edge cases

- **Trial ends mid-sentence:** the grace window lets the save land. If the user writes past the grace window, the draft stays in IndexedDB, the user is told plainly, and it syncs the moment they unlock. The Vault export includes unsynced local drafts, so even this text can leave with them.
- **Restore with an expired trial:** blocked for now (it writes entries). See Open questions.
- **Repeat trials with new emails:** accepted. Fighting it costs more than it loses.
- **Sealed letters after the trial:** they still unseal on their date. Reading is free forever.

10. Build phases for Claude Code

There are eleven phases, each shippable and testable on its own. Export comes early (phase 4) on purpose: the format gets locked before features pile on, and your own beta journal is protected from day one.

How to run each phase: give Claude Code this spec plus a CLAUDE.md, ask for that phase only, run its "done when" checks yourself, then commit before starting the next phase.

#	Phase	Builds	Done when	Model
0	Scaffold and look	Vite + React + TS, PWA setup, routing stubs for all nine screens, design tokens, fonts, desk and page CSS, both themes	App installs to a phone home screen; Lamplight and Daylight both render correctly on phone and desktop; an Arabic sample paragraph renders RTL inside an English page	Sonnet
1	Auth, schema, RLS	Supabase project, all migrations, signup trigger, `can_write()` and `can_edit()`, magic-link login	A test script proves user A can't read user B; inserts fail after the trial date is moved into the past; `licenses` can't be written from the client	Sonnet, with an Opus review of the RLS
2	Editor and autosave	Tiptap editor with auto direction, IndexedDB buffer, server sync, save status, multi-tab handling, `entry_versions` on concurrent edit	Pull the network mid-sentence, close the tab, reopen: every word is there. Two tabs editing one entry lose nothing.	Opus (this is where data loss hides)
3	Organization	Tracks, tags, chapters, templates including Spiritual, language tag	Create, rename, and assign each one; a chapter shows the right entries by date; the Spiritual template pre-fills	Sonnet

#	Phase	Builds	Done when	Model
4	Vault (text)	JSON export with `_readme`, restore dry run, merge apply, report, backup nudge	Round-trip test: export, wipe, restore, export again. The two files match apart from timestamps. Restoring the same file twice changes nothing.	Opus
5	Browse and read	Calendar with dots, list with filters, search (full-text plus trigram), Read mode, jump to date	Search finds partial words in English and Arabic; Read mode flips by day with swipe and arrow keys	Sonnet
6	Photos	Client compression, EXIF stripping, upload queue, storage RLS, 2 GB cap, .zip export and restore with media	A 12 MB phone photo stores at under 400 KB with no GPS data; the cap blocks cleanly; a .zip round-trips with photos	Sonnet
7	Letters	Letters track, "To" field, seal with unlock date, wax-seal card	A sealed letter can't be opened in the UI before its date, opens after it, and exports with `sealed: true`	Sonnet
8	Prompts and dice	Prompt table, seed-file loader, filters by depth, mood, and time, brass die	Loading your prompt JSON populates the table; the die never repeats a prompt within 20 rolls	Sonnet

#	Phase	Builds	Done when	Model
9	Licensing	Hosted checkout, webhook Edge Function, states and grace window, trial messaging, refund handling	In provider test mode: buy, unlock, refund, and read-only all behave as in section 9; replaying a webhook does nothing	Sonnet, with an Opus review of the webhook
10	Launch polish	Accessibility pass, privacy policy page, account deletion (export offered first), Sentry, landing page, empty states, microcopy pass	Keyboard-only use works end to end; contrast passes AA; deleting an account removes all rows and files	Sonnet

11. Risks and open questions

The biggest risk is economic, not technical: a one-time price pays for storage that lasts forever. The biggest technical risk is losing text, which is why phases 2 and 4 get the strongest model and the hardest tests.

Risks

Risk	Impact	Mitigation
Lifetime hosting on one-time revenue	Costs grow with every account that never leaves	2 GB photo cap; track cost per user monthly; revisit the $2.99/mo option once there are real numbers
Read-only accounts pile up forever	Storage paid for users who never pay	Dormancy policy (open question 2)
Autosave or merge bug destroys text	Fatal to the brand	IndexedDB buffer, `entry_versions`, round-trip tests, Opus on phases 2 and 4

Risk	Impact	Mitigation
Safari can clear IndexedDB for sites not installed to the home screen after several weeks of no use	The local buffer can vanish on iPhone	Treat the buffer as a safety net, not storage; sync early and often; encourage installing the PWA
Bidirectional text quirks in the editor (cursor jumps, selection across Arabic and English)	Arabic writers hit friction	Test real mixed Arabic and English text in phase 2, not at the end
Users assume sealed letters are encrypted	Trust damage	Copy states it plainly: "Sealed means hidden until the date, not encrypted."
Not end-to-end encrypted	Privacy-focused users object	Honest privacy policy; explain what we can and can't see; no analytics on entry content
Deleted data lingers in database backups	Deletion promise is weaker than it sounds	Disclose the backup retention window in the privacy policy
Supabase dependency	Lock-in for us, not the user	Standard Postgres plus the user export; migration is possible

Open questions (decisions for Yasin)

- [ ] **Restore after the trial ends.** Allowing it is maximally pro-ownership, but anyone could then edit their JSON in a text editor and "restore" new entries for free. Recommendation: block it, with a clear message that reading and export still work.
- [ ] **Dormant read-only accounts.** Options: (a) keep forever; (b) after 24 months of no sign-in, email twice with an export link, then delete; (c) after 24 months, keep text but delete photos. Recommendation: (c). It keeps the "reading is free forever" promise for words, which cost almost nothing.
- [ ] **Payment provider:** Lemon Squeezy or Paddle. Both are merchants of record; compare current fees and payout terms before phase 9.
- [ ] **Name check:** search trademarks and domains for "Daftar" (there may be existing apps using it).
- [ ] **Default tracks:** confirm the starting set (Daily, Spiritual, Letters, Dreams) and their colours.

- [ ] **Privacy policy:** who drafts it, and does it need a lawyer's review before launch?
- [ ] **Pricing model:** $49 one-time for now. Revisit a subscription after business planning; it changes the tagline.